```
        ordering = [ "name", ]
    ...

class Room(models.Model):
    ...
    class Meta:
        ordering = [ "name", ]
    ...
```

Exercise 3

Create `admin.ModelAdmin` classes for the `Venue` and `RoomModel` classes, including custom linked columns methods, like with `Musician` and `Band` as well as searchable name fields.

Class declarations for Django Admin models for `Venue` and `Room` objects are similar to your existing `Musician` and `BandAdmin` classes. The following listing shows the `VenueAdmin` and `RoomAdmin` class declarations.

Listing E.20 Django Admin classes for the `Venue` and `Room` classes

```
# RiffMates/bands/admin.py
...
@admin.register(Venue)
class VenueAdmin(admin.ModelAdmin):
    list_display = ("id", "name", "show_rooms")
    search_fields = ("name",)

    def show_rooms(self, obj):
        rooms = obj.room_set.all()
        if len(rooms) == 0:
            return format_html("<i>None</i>")

        plural = ""
        if len(rooms) > 1:
            plural = "s"

        parm = "?id__in=" + ",".join([str(b.id) for b in rooms])
        url = reverse("admin:bands_room_changelist") \
            + parm
        return format_html(
            '<a href="{}">Room{}</a>', url, plural)

    show_rooms.short_description = "Rooms"


@admin.register(Room)
class RoomAdmin(admin.ModelAdmin):
    list_display = (
        "id",
        "name",
    )
    search_fields = ("name",)
```

The .show_rooms() method is a custom column that displays a link to rooms associated with each Venue object.

If there are no rooms for this venue, the column shows "None".

If there is more than one room for this venue, append an s to the text, showing how many rooms there are.

Use the format_html() function to create a link that Django displays without escaping its HTML.

The reverse() function looks up a URL route; the result can have query parameters to filter the results displayed by the Django Admin.

Setting the .short_description member of the function changes the title of the associated column.